# Reinforcement Learning HW4 - Report

Batuhan Sencer

April 2026

## 1 Task 1

On the first given task I have implemented `add` and `calc_reward_to_go` in buffer class. Then implemented `collect_data`, `act`, and `rescale_actions`.

### 1.1 Function: `add`

We check if the buffer is oversize if we are not then it follows as: assign index to a variable called current_row, then assign the retrieved state, action, reward, and done to buffers' states, actions, rewards, and dones columns on index of current_row.

### 1.2 Function: `calc_reward_to_go`

We start by assigning 0 to `reward_return` to store the running total of future discounted awards. Then the loop starts backward throught the stored rows, because the return at each timestep depends on the rewards that come after it. If `self.dones[row]` is true, that shows the current row is the end of an episode, so we can reset the `return_reward` to 0. Then we use $current\_reward + \gamma * reward\_return$ to store the reward to go. Overall, the function fills `self.ret_to_go` with the total discounted reward starting from each timestep until the episode ends.

### 1.3 Function: `collect_data`

This functions purpose is to collect the required data form the environment. We collect the data by first creating a buffer sized for `size` transitions, then we reset the environment by `env.reset()` to receive the first observation. Then iterate for size times on each iteration we treat the current observation as the current state then we ask the agent for an action by `act(agent, observation)` then we send the received action to environment using `env.step(action)`. After receiving the data from the environment we check if the episode is done by checking `truncated` or `terminated` values, after this check we store `current_state, action, reward, done` in the buffer by calling `buffer.add()`. When the episode ends we reset the environment and start a new environment. If its not

ended then we simply continue form the next observation. At the end we return the filled buffer and the average reward per step with `np.mean(buffer.rewards)`

### 1.4 Function: `act`

The act function starts by getting policy, and state arguments, then we convert state variable to tensor format from numpy array format. After completing the conversion we get $\mu, \sigma$ from policy and the state. Then we call the `Normal` function to receive the normalized distribution.

### 1.5 Function: `rescale_actions`

The main job is to convert a normalized action to (-1,1) range so it can fit into the environment's actual action range [amin, amax].

## 2 Task 2

On task 2 I have compared $1e^{-5}$, $2e^{-4}$, and $9e^{-3}$ with $1e^{-4}$, and $3e^{-4}$. Result as follows:

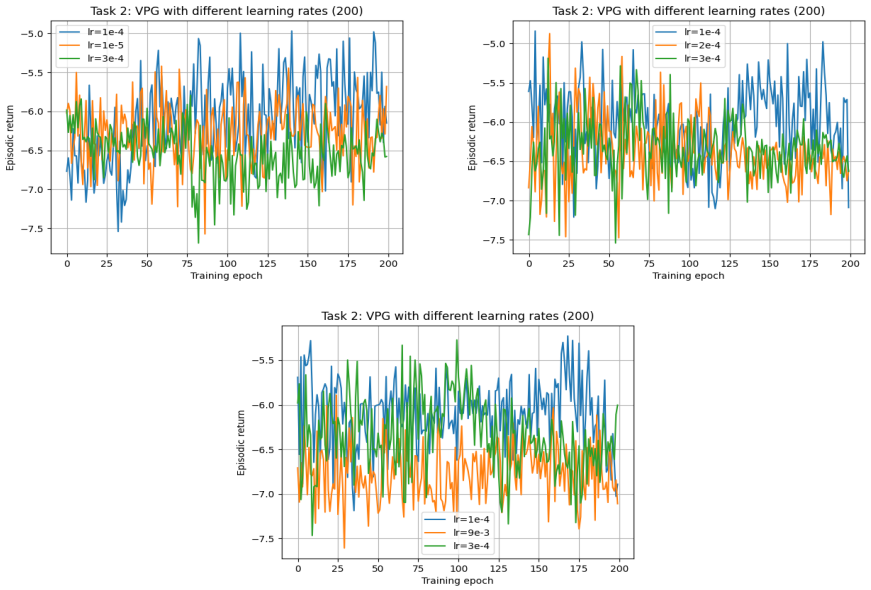
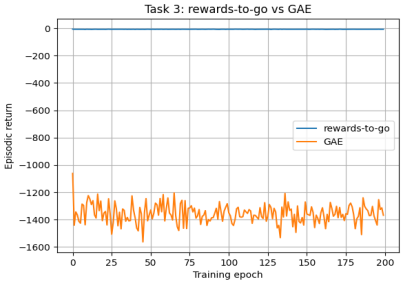



Figure 1: Task 2 results for different learning rates.

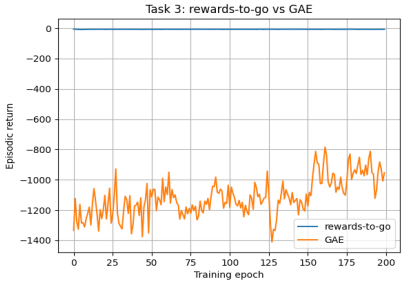
In Task 2, the step size has a clear effect on learning stability and convergence. When the learning rate is very small, such as $1e^{-5}$, the learning is more stable, but it improves very slowly because the policy updates are too small.

When the learning rate is too large, such as $9e^{-3}$, the curve becomes noisy and performs worse because the updates are too aggressive, so the policy keeps jumping around instead of converging smoothly. The $3e^{-4}$ learning rate also shows some noise and does not consistently reach the best episodic return. Overall, $1e^{-4}$ seems to perform the best because it reaches higher episodic return values compared to the other learning rates while still keeping the training more stable. This could be because $1e^{-4}$ gives a better balance between learning speed and stability: it is not too small to make learning very slow, and it is not too large to make the updates unstable.

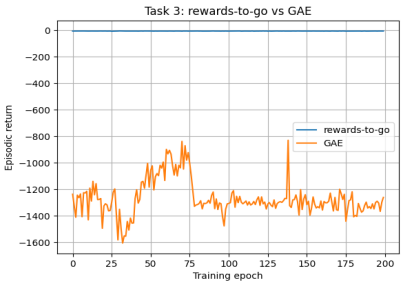
# 3 Task 3



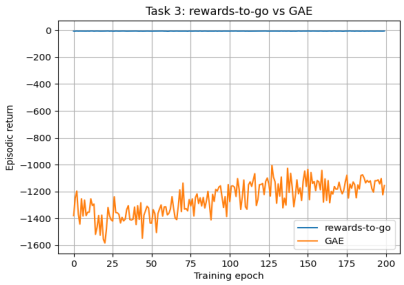

(a) Task 3 1e-2



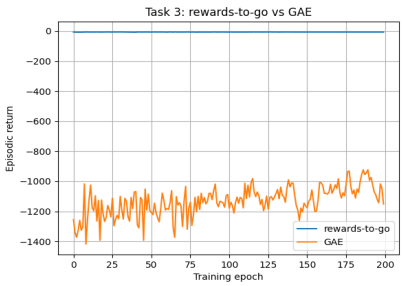

(b) Task 3 1e-3




(c) Task 3 1.7e-3




(d) Task 3 1e-4



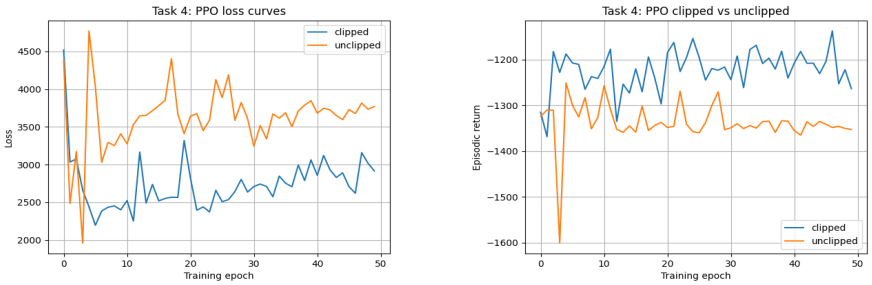

(e) Task 3 3e-4

Figure 2: Task 3 results for different learning rates.

In Task 3, we compared rewards-to-go and GAE with different learning rates. In Figure 2 The rewards-to-go line stays almost flat around zero in all graphs, while the GAE line changes more during training and shows the real behavior of the learning process. The noisiest result is $1.7e^{-3}$(Figure 2c) because the GAE curve suddenly improves around epochs 50–75, but then drops back down and becomes unstable again. The $1e^{-2}$(Figure 2a) learning rate also performs

poorly because it stays around very negative returns and does not show a clear improvement, which could mean the learning rate is too large and the updates are too aggressive. The best result seems to be $1e^{-3}$((Figure 2b)) because its GAE curve gradually improves over time and reaches better episodic return values near the end compared to the other learning rates. This could be because $1e^{-3}$ gives a better balance. It is large enough to learn faster, but not too large to make the training unstable.

# 4 Task 4



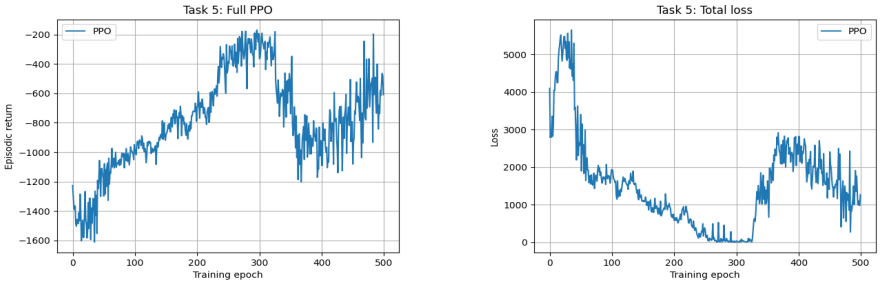

(a) Task 4 Loss

(b) Clipped vs. Unclipped

Figure 3: Task 4 Plots

In Task 4, the clipping is used in PPO to limit how much the policy can change in one update. If we do not use clipping, the update can become too large, and this can make the training unstable. In the loss plot, the unclipped version has higher loss values and also has a very big spike at the beginning. The clipped version has lower loss values and looks more controlled. In the episodic return plot, the clipped version performs better because it stays around $-1200$, while the unclipped version stays around $-1350$ and also drops a lot near the beginning. Because of this, clipping helps the model train more stable by stopping too aggressive policy updates. Overall, the clipped PPO gives better and more stable results than the unclipped PPO.

# 5 Task 5




(a) Full PPO (b) Total Loss

Figure 4: Task 5 Plots

Figure 5: PPO Strobe

In Task 5, the full PPO model performs better than the previous tasks. At the beginning, the episodic return starts around $-1500$, but during training it improves and reaches close to $-200$ to $-400$ at the best part. This shows that the agent learned the task better. However, the training is not fully stable because after around epoch 300, the return drops again, and then it starts to recover. The total loss also decreases a lot in the middle of training, but later it increases again, so this shows PPO is still sensitive to hyperparameters. GAE helps because it gives a better advantage estimate and makes the learning less noisy compared to using only rewards-to-go. Clipping helps because it limits how much the policy can change in one update, so it prevents very aggressive updates and makes training more stable. From the previous tasks, I observed that the step size is very important. If the learning rate is too large, the result becomes noisy and unstable, but if it is too small, the learning becomes slow. Overall, full PPO gives the best result because it uses both GAE and clipping together, but the plots still show that PPO depends a lot on good hyperparameter choices.